What is a Pointer?

A **pointer** is a variable that stores the **memory address** of another variable — not the value itself, but the **location** where the value is stored in memory.

> If you know C language — this is the exact same concept

File Setup

```
golang-course/
└── 15-pointers/
    └── ptr.go
```

```go
package main

import "fmt"

func main() {
    // code goes here
}
```

The Problem First — Pass By Value

Let's understand WHY pointers are needed:

```go
func changeNum(num int) {
    num = 5
    fmt.Println("in changeNum:", num)
}

func main() {
    num := 1
    changeNum(num)
    fmt.Println("after change in main:", num)
}
```

Output:

in changeNum: 5
after change in main: 1 ← NOT changed!

Why didn't it change?

Because Go passes arguments **by value** — meaning:

- When you pass `num` to `changeNum()`
- Go creates a **complete copy** of `num`
- The function works on that **copy**
- Original `num` in `main()` is **never touched**

MEMORY:

```
MEMORY:
┌──────────────┐      ┌──────────────┐
| num = 1      |      | num = 1      | ← copy created
| (main)       |      | (changeNum)  | ← this gets changed to 5
└──────────────┘      └──────────────┘

      ↑                    ↑
  original            copy — original never affected
```

Seeing the Memory Address

Before learning pointers, let's see what a memory address looks like:

num := 1
fmt.Println(&num) // prints memory address

Output:

0xc0000b4008 ← this is the memory address

- `&` symbol before a variable = **get its memory address**
- The address looks like a hexadecimal number
- This is the exact location in RAM where `num` is stored

The Solution — Pointers (Pass By Reference)

Instead of passing the value, pass the **memory address**:

```
func changeNum(num *int) {  // *int = pointer to integer
    *num = 5                // dereference and change value
    fmt.Println("in changeNum:", *num)
}

func main() {
    num := 1
    changeNum(&num)         // pass memory address
    fmt.Println("after change in main:", num)
}
```

Output:

```
in changeNum: 5
after change in main: 5  ← changed!
```

Two Key Symbols — & and *

These two symbols are the heart of pointers:

& — Address Of Operator

```
num := 1
fmt.Println(&num)  // 0xc0000b4008 — gives address of num
```

- Put & before a variable
- Returns the **memory address** of that variable
- Used when **passing** a pointer to a function

* — Two uses:

Use 1 — Declaring a pointer type:

```
func changeNum(num *int) {
```

```
//              ↑
//        *int means "pointer to integer"
//        this parameter receives a memory address
```

Use 2 — Dereferencing (getting/setting value at address):

```
*num = 5
//↑
// * before pointer variable = go to that address and change value there
```

Breaking Down the Full Pointer Code

```
func changeNum(num *int) {
    *num = 5
    fmt.Println("in changeNum:", *num)
}
```

Part	Meaning
`num *int`	parameter receives a pointer to an integer
`*num = 5`	go to the address stored in `num`, put 5 there
`*num` in Println	go to the address, read the value there

```
func main() {
    num := 1
    changeNum(&num)   // pass address of num
    fmt.Println(num)  // 5 — original changed!
}
```

Part	Meaning
`num := 1`	create variable with value 1
`&num`	get memory address of num
`changeNum(&num)`	pass address to function

`fmt.Println(num)`	prints 5 — original was changed

Memory Visualization

```
## Memory Visualization

### Pass By Value (no pointer):
```
main memory: changeNum memory:
┌──────────┐   ┌──────────┐
| num = 1 | copy→ | num = 1 |
└──────────┘ | num = 5 | ← only copy changed
      ↑   └──────────┘
  still 1!   deleted after function ends
```

### Pass By Reference (with pointer):
```
main memory:
┌──────────────────┐
| num = 1 |
| address:0xc000 |◄──────────────┐
└──────────────────┘ |
                                   |
changeNum receives: 0xc000 |
→ goes to that address ────────────┘
→ changes value there to 5
→ main's num is now 5!
```
```

Why is Dereferencing Needed?

When the function receives a pointer:

```
func changeNum(num *int) {
    // num here is the ADDRESS — like 0xc000
    // NOT the value

    num = 5   // ❌ ERROR — you're trying to put 5 into an address variable
              //          doesn't make sense

    *num = 5  // ✅ CORRECT — go to that address, put 5 there
}
```

- `num` = the address itself (like a house number)
- `*num` = the value AT that address (like what's inside the house)

Real Life Analogy

Think of it like a **Google Maps location**:

- `num = 1` → the actual person living at a house
- `&num` → the GPS coordinates / address of that house
- `*int` → "I accept GPS coordinates"
- `*num = 5` → "go to those coordinates and change who lives there"

Without pointer → you get a **photocopy** of the person — changing the copy doesn't affect the original.

With pointer → you get the **actual address** — you go there and change the actual person.

Complete Code Example

```
package main

import "fmt"

// Pass by value — original NOT changed
```

```go
func changeNumByValue(num int) {
    num = 5
    fmt.Println("in changeNum (value):", num)
}

// Pass by reference — original IS changed
func changeNumByRef(num *int) {
    *num = 5
    fmt.Println("in changeNum (ref):", *num)
}

func main() {

    // Pass by value demo
    num1 := 1
    changeNumByValue(num1)
    fmt.Println("after value change:", num1)  // still 1

    // See memory address
    num2 := 1
    fmt.Println("memory address:", &num2)  // 0xc000...

    // Pass by reference demo
    num3 := 1
    changeNumByRef(&num3)                    // pass address
    fmt.Println("after ref change:", num3)        // 5 — changed!
}
```

Output:

```
in changeNum (value): 5
after value change: 1       ← not changed

memory address: 0xc0000b4008

in changeNum (ref): 5
after ref change: 5         ← changed!
```

Running the Program

go run 15-pointers/ptr.go

Summary Table

Symbol	Name	Use
`&variable`	Address of	Get memory address of variable
`*Type`	Pointer type	Declare a pointer parameter
`*pointer`	Dereference	Read or write value at address

When Will You Use Pointers?

The instructor mentions these use cases coming up:

Use Case	Why Pointer Needed
Structs passed to functions	Modify original struct, not a copy
Buffers	Pass buffer so inner function can fill it
Large data structures	Avoid copying huge amounts of data
Methods that modify state	Change the actual object, not a copy

---

Pass By Value vs Pass By Reference

Feature	By Value	By Reference (Pointer)
What is passed	Copy of value	Memory address
Original changed?	❌ No	✅ Yes
Syntax	`func f(n int)`	`func f(n *int)`

Calling	`f(num)`	`f(&num)`
Inside function	`n = 5`	`*n = 5`
Memory usage	More (copy created)	Less (just address)

Key Takeaways

1. A **pointer** stores the memory address of a variable — not the value
2. By default Go passes arguments **by value** — a copy is made
3. Changing a copy does **NOT** affect the original
4. Use `&variable` to get the **memory address** of a variable
5. Use `*Type` in function parameter to **receive a pointer**
6. Use `*pointer` inside function to **read or change** the value at that address
7. This is called **dereferencing** — going to the address and accessing the value
8. Pass by reference means the **original variable gets modified**
9. Pointers are essential when working with **structs, buffers, and large data**
10. This concept is the **same as C language** — if you know C, you already know this

One Thing I Could Have Explained MORE Clearly

The transcript specifically shows this error moment:
go
```
func changeNum(num *int) {
    num = 5   // ❌ compiler complains here
}
```
The instructor explains — when you receive a pointer and try to directly assign without *, compiler says "cannot use num" — because num here is an address, not an integer. You cannot put integer 5 into an address variable.
go
```
// num holds something like 0xc000b4008
// doing num = 5 is like saying
// "put number 5 into a GPS coordinate" — makes no sense

*num = 5  // correct — go TO that GPS coordinate, put 5 there
```

Also This Moment From Transcript
When printing inside the pointer function:
go
```
func changeNum(num *int) {
```

```
    *num = 5
    fmt.Println(*num)  // need * here too when READING
}
```

- Without * → prints the address like 0xc000b4008
- With * → prints the actual value like 5